

可变分辨率 Chunk 筛选的 GPU 优化建议

聚焦 `weighted_select(1).cu` 中第 11-17 条与第 20 条改进方向

文档范围：仅讨论 GPU/CUDA 执行层优化，不改变当前“按 score 降序、按 clipped length 消耗 token budget、逻辑 leaf index 保证稳定 tie”的选择语义。重点覆盖：单 CTA 并行度、shared atomic、threshold bucket 压缩、radix threshold 查找、BlockScan、fallback pass 数、fast/fallback 统一，以及 interval 直接下传。

1. 当前选择器的执行结构与主要瓶颈

当前 fast path 在 `capacity <= 4096` 时，由 `weighted_prefix_fast_kernel<<<1, 1024>>>` 单 CTA 完成：先对 ordered FP32 score 的最高 8 bit 做按 token length 加权的 256-bin histogram，再把 threshold high-byte bucket 中的 leaf 压入 shared candidate buffer，随后只对该 bucket 做剩余 3 个 byte 的 radix refinement，最后按逻辑 leaf 顺序进行稳定 tie 处理和 `selected_prefix` 生成。`capacity > 4096` 时进入 fallback，使用 score 32 bit + leaf index 16 bit 的 48-bit stable key，执行 6 次 byte radix pass。

这套实现已经避免了对全部 leaf 做 full stable sort，算法方向是正确的；后续优化的重点应放在 GPU 利用率、原子竞争、中间扫描次数以及 fast/fallback 两套路径的冗余上。

```
Fast path (capacity <= 4096)
all leaves -> weighted high-byte histogram
            -> locate threshold bucket
            -> pack threshold-bucket leaves
            -> 3 radix refinements
            -> logical-order stable scan
            -> selected_prefix

Fallback (capacity > 4096)
all leaves -> 6 full weighted radix passes on 48-bit key
            -> logical compaction -> selected_prefix
```

2. 优化 11：单 CTA Selector 改为批量 Multi-CTA 执行

现状：`launch_prefix()` 对一次 selector 固定使用 `weighted_prefix_fast_kernel<<<1, 1024>>>` 或 `weighted_prefix_fallback_kernel<<<1, 1024>>>`。也就是说，一次选择过程最多占用一个 SM 上的一个 CTA；即使 GPU 有几十到上百个 SM，单个 selector 也不能横向铺开。

```
weighted_prefix_fast_kernel<<<1, 1024, 0, stream>>>(...)
weighted_prefix_fallback_kernel<<<1, 1024, 0, stream>>>(...)
```

如果上层执行模式是“每个 request/head/query 独立调用一次 selector”，则大量小型单 CTA kernel 会造成 GPU device occupancy 偏低，尤其在 decode 或多 head 场景中，瓶颈可能从纯计算变成 kernel 调度和单 CTA 延迟。

2.1 建议方案

将调用接口改成 batch selector: grid.x 对应 selector 实例, 而不是每个实例单独 launch。每个 blockIdx.x 仍使用一个 1024-thread CTA, 保持现有 block 内算法基本不变。

```
grid.x = num_selectors

selector = blockIdx.x
scores = scores_base + selector * score_stride
lengths = lengths_base + selector * length_stride
starts = starts_base + selector * start_stride
...

// CTA 0 -> request/head/query 0
// CTA 1 -> request/head/query 1
// CTA 2 -> request/head/query 2
// ...
```

这样做的优点是算法风险最低: weighted radix 的 exactness、稳定 tie 和 token-budget 语义完全不变, 只是把多个独立 selector 合并到一个 grid 中。对于 head 数、batch 数或并发 request 较高的场景, 可以显著提高 SM 覆盖率。

优先级: P1。若真实调用层确实存在大量独立 selector, 这一项可能是整个文件中收益最高的系统级优化; 如果当前 selector 本来就是全局只调用一次, 则收益有限, 应先 profile 调用拓扑。

3. 优化 12: shared histogram atomic 改为 Warp-Aggregated Histogram

现状: Phase A 以及后续 radix refinement 都通过 `atomicAdd(&histogram[bin], length)` 累加 token weight。注意这里累加的是 clipped leaf length, 而不是 leaf count, 因此当 score 的高位分布集中时, 多个 lane 会同时竞争少数几个 shared-memory bin。

```
bin = ordered_float_key(scores[index]) >> 24;
atomicAdd(&histogram[bin], length);
```

attention/summary score 往往并非均匀覆盖 FP32 ordered-key 的 256 个高位桶, 而是聚集在一个较窄范围内。因此 shared-memory atomic 的热点竞争可能明显, 尤其第一阶段所有 leaf 都参与统计。

3.1 建议方案

在 warp 内先对相同 bin 的 lane 做聚合, 再由每个 distinct bin 的 leader 执行一次 `atomicAdd`。CUDA 上可以利用 `__match_any_sync` 获取相同 bin 的 lane mask, 再通过 warp reduction 汇总 length。

```

unsigned mask = __match_any_sync(0xffffffff, bin);
int leader = __ffs(mask) - 1;
uint32_t warp_sum = reduce_length_over_mask(length, mask);
if (lane == leader) {
    atomicAdd(&histogram[bin], warp_sum);
}

```

若一个 warp 中 20 个 lane 落在同一 bin，原来需要 20 次 shared atomic，新实现只需要 1 次。对 clustered-score workload，原子数量可大幅下降。对于分布很散的情况，额外 match/reduction 会有一定固定成本，因此建议保留两个版本并用真实 score 分布 benchmark。

验证指标：统计每次 Phase A 的 unique bins/warp、shared atomic 次数，以及 threshold high-byte bucket ratio。若每 warp unique bin 数远小于 32，则 warp-aggregated histogram 很值得做。

4. 优化 13: candidate_count 的逐 leaf atomicAdd 改为 Warp Compaction

Phase B 会重新扫描所有 leaf，只把命中 threshold high-byte 的 leaf 压入 threshold_candidates。当前每个命中 leaf 都执行 atomicAdd(&candidate_count, 1)，当 threshold bucket 较大时，这个单一 shared counter 会成为热点。

```

if ((key >> 24) == threshold_byte) {
    int pos = atomicAdd(&candidate_count, 1);
    threshold_candidates[pos] = pack(length, index);
}

```

4.1 建议方案

使用 warp ballot + prefix：每个 warp 先得到 hit mask，计算本 warp 的 hit_count，仅 warp leader 对 candidate_count 做一次 atomicAdd 获取 base position，然后每个命中 lane 用 popc(mask & lanemask_lt) 得到 warp 内偏移。

```

bool hit = ((key >> 24) == threshold_byte);
unsigned mask = __ballot_sync(0xffffffff, hit);
int warp_count = __popc(mask);
int base = 0;
if (lane == 0 && warp_count > 0)
    base = atomicAdd(&candidate_count, warp_count);
base = __shfl_sync(0xffffffff, base, 0);
int local = __popc(mask & ((1u << lane) - 1));
if (hit)
    threshold_candidates[base + local] = pack(length, index);

```

这样 atomic 数量从“每个 candidate 一次”变成“每个有 candidate 的 warp 一次”，最坏也约减少 32 倍。实现仍保持 candidate 内容完全相同，不改变后续 radix refinement。

5. 优化 14：256-bin 阈值查找由 thread 0 串行扫描改为并行 Prefix/Scan

现状：每次 histogram 构建结束后，都由 tid==0 从 bin 255 向 0 顺序累加 above，直到发现 $\text{above} + \text{histogram}[\text{bin}] > \text{remaining}$ 。fast path 需要执行 4 次这样的查找，fallback 需要执行 6 次。单次只有 256 项，绝对开销不算大，但它是 block 内完全串行区，会让其余 1023 个线程等待。

5.1 建议方案

用 256 个线程各负责一个 bin，对 histogram 做 reverse exclusive prefix sum。令 $\text{above}[\text{b}] = \text{sum}(\text{hist}[\text{j}], \text{j} > \text{b})$ ，则 crossing bin 满足 $\text{above}[\text{b}] \leq \text{remaining} < \text{above}[\text{b}] + \text{hist}[\text{b}]$ 。之后通过 ballot/ffs 找到唯一 crossing bin。

```
// tid 0..255 owns one bin
w = histogram[255 - tid];
reverse_prefix = BlockScan256(w);

cross = (reverse_prefix <= remaining) &&
        (remaining < reverse_prefix + w);

mask = __ballot_sync(..., cross);
chosen_lane = __ffs(mask) - 1;
chosen_bin = 255 - chosen_lane;
```

建议不要为这一项单独引入复杂的多 CTA 算法；256 项很小，最佳实现应该仍然在一个 CTA 内完成。它更适合作为“低风险微优化”，在 histogram atomic 优化之后再评估。

6. 优化 15：Final Logical Scan 的两次 BlockScan 合并为一次 Pair Scan

当前 fast path 的最终 logical-order pass 对每个 1024-leaf stripe 执行两次 CUB BlockScan。第一次扫描 equal_length，用于处理 threshold_key 上的稳定 index tie；第二次扫描 selected_length，用于生成 selected_prefix。

```
BlockScan(equal_length)    -> equal_exclusive / equal_aggregate
... compute selected_length ...
BlockScan(selected_length) -> exclusive / aggregate
```

这两个 scan 之间存在依赖，但可以通过重新定义扫描状态合并。对每个 leaf 定义 higher_length 与 equal_length：higher 表示 $\text{key} > \text{threshold_key}$ 的完整长度，equal 表示 $\text{key} == \text{threshold_key}$ 的完整长度。一次扫描同时维护二元组 (H,E)。

6.1 一次 Pair Scan 的计算方式

```
higher_i = (select_all || key > threshold_key) ? length : 0;
equal_i  = (!select_all && key == threshold_key) ? length : 0;

exclusive scan on pair<int,int>:
    (H_excl, E_excl)

selected_length = higher_i;
if (equal_i > 0) {
    selected_length = min(max(remaining - E_base - E_excl, 0), length);
}

selected_prefix = H_prefix + min(E_prefix, remaining);
```

更进一步，可以直接让 scan 的输出产生“截至当前 leaf 的 selected token prefix”，无需再对 selected_length 做第二次 BlockScan。这样每个 stripe 只保留一次同步密集型 collective，减少 shared TempStorage 重用和 __syncthreads 次数。

风险：实现时要特别验证跨 stripe 的 E_base/H_base 更新，以及 equal-score leaf 在多个 stripe 中的稳定 index 顺序。建议逐 bit 对照现有 fast path 输出。

7. 优化 16：Fallback 从 6 次 Full Radix Pass 降为 4 次

当前 fallback 为保证 (score descending, leaf index ascending) 的稳定排序语义，把 ordered FP32 score 32 bit 与 16-bit leaf tie 合并成 48-bit stable_leaf_key，因此需要 6 个 byte pass。

```
stable_leaf_key(score, index)
= [ ordered_score:32 ][ 0xffff-index:16 ]

48 bit / 8 bit per pass = 6 passes
```

但 fast path 已经证明：leaf index tie 不需要进入 radix key。只要先用 32-bit score 找到 threshold_score，最后按逻辑 leaf index 顺序扫描 equal-score leaves，就可以精确实现相同的稳定语义。

7.1 新 fallback

```
Pass 0: score bits 31..24
Pass 1: score bits 23..16
Pass 2: score bits 15..8
Pass 3: score bits 7..0
        -> exact threshold_score

Final logical scan:
key > threshold_score -> full leaf
key == threshold_score -> earlier leaf index first
                        crossing leaf partial
```

这样 capacity > 4096 时，每个 leaf 的全量读取/clip/key 计算由 6 次降到 4 次，理论上 radix 主循环的全量扫描工作量减少约 33%。由于 tie 顺序仍由最终 logical scan 精确实现，选择结果可以保持 bit-identical。

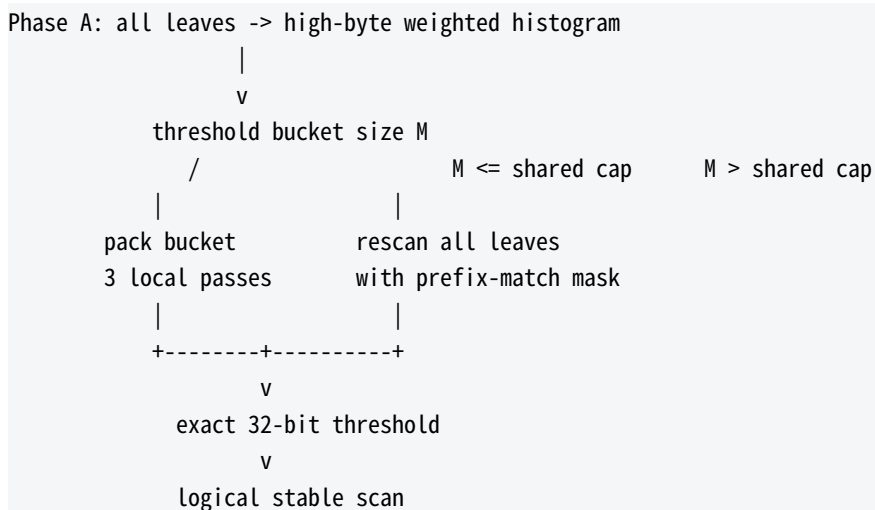
优先级： P1，推荐直接实施。它是当前 fallback 中最清晰的“少做两遍全量扫描、且不改变算法结果”的优化。

8. 优化 17：Fast / Fallback 统一为 Hybrid Weighted Radix

当前实现以 capacity ≤ 4096 为分界：小容量走“高位 bucket pack + 3 次 bucket refinement”，大容量直接走 6-pass fallback。这导致两套核心 selector 逻辑长期并存，维护成本高，也没有利用“大容量但 threshold bucket 很小”的情况。

8.1 统一策略

第一步对所有容量都执行相同的最高 8-bit weighted histogram，得到 threshold bucket 和其 candidate_count M。随后根据 M，而不是 capacity，决定是否 pack。



关键变化是：kFastMaxLeaves 不再表示“整个 selector 最大容量”，而只表示“能放进 shared candidate buffer 的 threshold bucket 最大大小”。因此即使 capacity=20K，只要 threshold high-byte bucket 只有几百个 leaf，仍可以走 fast packed refinement。反之，即使 capacity<4096，但 score 高位过度集中导致 bucket 过大，也可以自动退化为 prefix-rescan。

这一设计还能直接利用代码现有的 profile_stats：candidate_count/n 的分布已经被统计出来，可以先用线上/离线 profile 判断 8-bit first radix 是否足够。如果 threshold bucket 经常占 30%-60%，可以进一步测试 10-bit 或 11-bit first radix，以更多 shared histogram 空间换更小的后续 candidate set。

收益： 统一后可以删除大部分独立 fallback 逻辑，减少维护分叉；同时性能决策从静态 capacity 阈值变成动态 bucket density，更符合真实 score 分布。

9. 优化 20：保留 Interval 表示，避免再展开为 Token ID

当前 weighted selector 先输出 selected_prefix，随后 expand_selected_kernel / expand_intervals_kernel 将选中的连续区间展开成 candidates[budget]，即每个 token 一个 int32 ID。对 variable-resolution chunk 而言，这一步实际上把前面已经获得的“连续区间结构”重新拍扁。

```
Selected variable leaves
  (start, taken_length)
    |
    v
expand kernel
  |
  v
[token_id, token_id, token_id, ...] // budget entries
  |
  v
fine scoring / sparse attention
```

如果后续 fine QK / gather kernel 可以直接消费 interval，则可以保留紧凑表示：每个选中 leaf 仅传 (start, taken_length, output_offset)，sink 和 tail 也自然是 interval。这样可以直接消除 token ID materialization。

9.1 推荐接口

```
struct SelectedInterval {
    int32_t start;
    int32_t length;
    int32_t output_offset; // optional
};

// downstream
for each interval assigned to a warp/CTA:
    for token = start + lane; token < start + length; token += warpSize:
        load K/token metadata
        compute fine QK / gather / attention
```

带宽上，原实现至少需要写 budget * 4B 的 candidate IDs，后续 kernel 再读 budget * 4B；interval 方式只需读写 O(num_selected_leaves) 个三元组。当 budget 很大而 selected leaf 数远小于 budget 时，这部分 metadata traffic 可以显著下降。更重要的是，paged-KV 地址计算也可以按 interval 顺序展开，改善访问连续性。

9.2 与 crossing leaf 的兼容性

当前 crossing leaf 本来就是“只取该 leaf 的前 taken 个 token”，因此 interval 可以精确表示为 (first, taken)。如果未来进一步实现 threshold-near hierarchical refinement，也仍然可以自然输出多个更细的 interval，无需重新恢复为离散 token ID。

优先级： P1/P2，取决于后续 fine-scoring kernel 是否容易改接口。若 downstream 已经按照

candidates[] 做大量随机 gather，则需要联合改造；如果 downstream 本身可以按连续 token 段处理，这一项的系统收益通常很高。

10. 推荐实施顺序

顺序	优化项	结果一致性	代码侵入性	建议
1	Fallback 6 pass -> 4 pass	可完全一致	低	先做，收益清晰
2	candidate_count warp compaction	可完全一致	低	原子热点优化
3	warp-aggregated histogram	可完全一致	中	需按 score 分布验证
4	两次 BlockScan -> 一次 Pair Scan	可完全一致	中	同步/collective 优化
5	Hybrid fast/fallback unified radix	可完全一致	中高	统一维护与自适应性能
6	batched multi-CTA selector	可完全一致	接口级	取决于上层并发结构
7	interval 直接下传	可完全一致	较高	需联合 downstream kernel
8	256-bin parallel threshold lookup	可完全一致	低	最后做微优化

11. 总结

这 8 项改进的共同目标不是改变 variable-resolution chunk 的选择准则，而是让 “weighted exact threshold selection” 更符合 GPU 的执行特点：用更高的 CTA 级并行度填满 GPU，用 warp 内聚合减少原子竞争，用 selection 而不是冗余 radix pass 减少全量扫描，用一次 collective 代替两次 collective，并尽可能保留 interval 结构避免 metadata 的展开和回读。

其中最值得先落地的是 6-pass -> 4-pass、candidate warp compaction 和 batch selector；如果后续允许联合修改 fine-scoring kernel，则 interval 直接下传会把可变分辨率 chunk 的结构优势真正延伸到下游计算，而不是在 selector 结束后重新退化成 token ID 列表。